

Phase 1: Requirements, Architecture, & Design Authority

Timeline: February 5 - 6 (11:59pm)

Assigned: Donzyl

You are responsible for defining what the system is, what it must do, and how it is structured.

1. Define the System Scope

- Write the problem statement (1–2 sentences)
 - Identify stakeholders and their expectations
 - Write assumptions & constraints:
 - Localhost only
 - Java sockets
 - Deterministic election (lowest ID)
 - Monitor is authority
 - No real persistence
-

2. Functional Requirements (FRs)

Write clear, testable FRs:

- Client request handling
- Primary election
- Automatic failover
- Heartbeat detection
- Client rediscovery
- Logging
- Failure simulation
- Admin actions

Each must be phrased as: “The system shall...”

3. Non-Functional Requirements (NFRs)

Define measurable constraints:

- Heartbeat detection threshold
 - Failover recovery time
 - Availability expectations
 - Maintainability
 - Observability (logging clarity)
-

4. Architecture Decisions

Lock in:

- Monitor responsibilities
- Server responsibilities
- Election strategy

- Role transitions
 - Message categories (heartbeat, promote, request, response)
-

5. UML Diagrams

Create:

- Class Diagram
- Sequence Diagram: Startup & Election
- Sequence Diagram: Client Request (normal + failover)

These diagrams are now contractually binding for the rest of the team.

6. Deliverables

You must deliver:

- Requirements document
- UML diagrams (PNG/PDF)
- Written architecture summary

Phase 2: Monitor & Election Logic

Timeline: February 7 - 8 (11:59pm)

Assigned: Egg

You are responsible for implementing the monitor, which acts as the single source of truth for server roles.

1. Monitor Server

- Accept server registrations
 - Track:
 - Server ID
 - Host/port
 - Last heartbeat timestamp
 - Current role (primary/backup)
-

2. Heartbeat Handling

- Receive heartbeats
 - Update timestamps
 - Detect timeouts using NFR thresholds
-

3. Failure Detection

- Detect missing primary heartbeats
 - Trigger election process
-

4. Deterministic Election

- Select alive backup with lowest ID

- Ensure only one promotion occurs
 - Prevent split-brain
-

5. Promotion Messaging

- Send PROMOTE message
 - Update internal state
 - Log all decisions with timestamps
-

6. Validation Checklist

Before handoff:

- Kill primary → monitor detects failure
 - Monitor promotes correct backup
 - Logs clearly show detection + promotion
-

7. Deliverables

- Working Monitor service
- Clear logs demonstrating election

Phase 3: Server Core & Failover Behaviour

Timeline: February 9 - 10 (11:59pm)

Assigned: Jake

You are responsible for building the server processes that actually perform the work and change roles.

1. Server Base Class

- Socket handling
 - Thread pool
 - Clean startup/shutdown
 - Connection handler abstraction
-

2. Heartbeat Sender

- Periodic heartbeat thread
 - Register with monitor
 - Handle delays cleanly
-

3. Primary vs Backup Logic

- Primary accepts client requests
 - Backup ignores client requests
 - Backup listens for PROMOTE
-

4. Promotion Handling

On PROMOTE:

- Switch role
 - Start accepting clients
 - Update logs
-

5. Failure Safety

- Graceful shutdown
 - No crash on monitor disconnect
 - Predictable role state
-

6. Validation Checklist

Before handoff:

- Start 3 servers
 - Kill primary
 - Backup becomes primary automatically
 - Clients can reconnect
-

7. Deliverables

- Server code
- Logs proving failover works
- Notes on server lifecycle

Phase 4: Client & Discovery Logic

Timeline: February 11 - 11 (11:59pm)

Assigned: Alashir

You are responsible for ensuring the client never gives up, even when servers die

1. Monitor Query

- Ask monitor for current primary
 - Parse host/port
-

2. Request Handling

- Send PROCESS / PING
 - Receive response
 - Display output
-

3. Failure Handling

- Detect socket failures
 - Retry loop
 - Re-query monitor
 - Reconnect automatically
-

4. Logging

- Connection attempts
 - Failures
 - Successful retries
-

5. Validation Checklist

Before handoff:

- Client runs during primary crash
 - Client reconnects without restart
 - No hard failures
-

6. Deliverables

- Client source code
- Usage instructions
- Logs showing reconnection

Phase 5: Logging, Scripts, Testing & Packaging

Timeline: February 12 - 13 (11:59pm)

Assigned: Massimo

You are responsible for making the system gradable, testable, and reproducible.

1. Logging Standardization

- Timestamps everywhere
 - Clear event labels:
 - Startup
 - Heartbeat
 - Failover
 - Client request
 - Recovery
-

2. Scripts

- run.sh / run.bat
 - kill-primary.sh
 - delay-heartbeat.sh
-

3. Execute Test Scenarios

- Normal operation
- Primary crash
- Backup crash
- Simultaneous failures
- Network delay

- Recovery

Record:

- Event time
 - Detection time
 - Failover time
 - Observed issues
-

4. Deliverables

- README instructions
- Test results section
- ZIP structure verified
- Final demo run